

Deep in the Linux Kernel

Abstract

This paper introduces an innovative architectural paradigm for operating system design, proposing the integration of deep learning models as loadable kernel modules (LKMs) within the Linux kernel to enable real-time artificial intelligence capabilities. Our prototype demonstrates a hybrid architecture comprising specialized LKMs for low-latency AI processing and a userspace orchestration system for managing high-level cognitive tasks. This design facilitates efficient multi-modal AI processing while preserving system stability and performance. We present the evolution of the system from a rudimentary LKM to a sophisticated module capable of executing tensor computations. While our current implementation focuses on basic AI inference within kernel space through custom system calls, we delineate potential future developments, including advanced knowledge transfer mechanisms between kernel and user space, dynamic LKM adaptation based on workload patterns, and seamless integration with hardware accelerators. We also elucidate challenges and open questions regarding security, scalability, and the trajectory towards more advanced artificial general intelligence (AGI) systems. This work establishes the foundation for a new generation of intelligent operating systems, catalyzing further research and development in this promising domain.

Keywords: *operating systems, artificial intelligence, loadable kernel modules, deep learning, real-time processing, system architecture, Linux kernel, AGI systems*

1 Introduction

A Loadable Kernel Module (LKM) [3] is a piece of code that can be dynamically loaded into or unloaded from the kernel at runtime. LKMs extend kernel functionality without requiring a reboot or recompilation of the kernel. A very simple LKM could be logging a message when it is loaded and unloaded. By dynamically loading and unloading code at runtime, LKMs enable developers to introduce new capabilities, such as device drivers [7], file system enhancements, or networking protocols, while maintaining system stability [4] and reducing downtime. Leveraging LKMs for computational purposes represents

a cutting-edge evolution of this flexibility, as it enables low-latency, high-performance operations directly within the kernel space. This approach minimizes the overhead of user-kernel context switching, allowing computationally intensive tasks, such as real-time data processing [6], cryptographic operations, or AI inference, to execute closer to the hardware. By exploiting the kernel’s proximity to system resources, LKMs can optimize the utilization of modern hardware accelerators, such as GPUs [5] and TPUs, while maintaining tight control over resource allocation and isolation. This paradigm is transformative for domains like edge computing, high-speed networking, and intelligent resource management, where computational demands intersect with stringent performance and reliability requirements.

LKMs can also function as modular nodes specialized for vision, language, or audio processing, communicating through high-speed kernel-level networking to form distributed frameworks. However, integrating multi-modality outputs requires a userspace orchestrator to unify data and manage high-level decisions. While LKMs are ideal for lightweight, modality-specific tasks, their static runtime nature and kernel memory constraints necessitate offloading large-scale computations, dynamic updates, and learning tasks to user-space components or distributed systems. Adopting a hybrid architecture with LKMs for real-time operations and userspace orchestration ensures efficient resource utilization, adaptability, and scalability across diverse computational applications. The deployment of LKMs in modern computing systems presents unique security and performance trade-offs that must be carefully balanced. While kernel modules operate with elevated privileges and unrestricted access to system resources, this power comes with inherent risks and responsibilities. The challenge lies in implementing robust isolation mechanisms and security boundaries while maintaining the performance advantages of kernel-space execution. Recent advances in hardware-assisted virtualization, memory protection keys, and secure enclaves have enabled new approaches to hardening LKM implementations without significantly compromising their computational efficiency. These developments have made it increasingly viable to leverage LKMs for security-sensitive applications while maintaining the strict isolation guarantees demanded by contemporary

computing environments.

2 Related Works

The collected papers provide significant insights into integrating AI into system-level functionalities. Marinoni et al. [2] explore real-time AI applications on heterogeneous platforms using hypervisors, but their focus is limited to AI at the middleware layer, not deep kernel-level integration. Jia et al. [8] emphasize modular and adaptable operating systems with AI agents but lack the real-time focus and deep system control envisioned in your work. Buttazzo [1] addresses the trust and safety challenges of AI in embedded systems but primarily within RTOS, without considering advanced AI use cases like anomaly detection or hardware-level optimization. Pan et al. [8] examine AI for networking tasks, such as packet classification, demonstrating the effectiveness of AI-powered stacks, yet they don't extend this to broader kernel functionalities like scheduling or resource management. Zhang et al. [8] provide a general review of AI enhanced operating systems, highlighting modular designs and AI for malware detection but without detailed implementations for high performance hardware accelerators or predictive tuning. Our paper uniquely advances this field by proposing direct integration of deep learning models into the Linux kernel as loadable modules (LKMs). This approach ensures ultra-low latency decision-making, real-time anomaly detection, and predictive kernel tuning, addressing system-wide challenges in networking, storage, and edge AI use cases. Unlike existing works, our approach leverages the kernel's proximity to hardware for direct utilization of accelerators (e.g., GPUs, TPUs) and achieves dynamic system adaptability, redefining operating systems as intelligent, self-tuning entities capable of revolutionizing performance critical domains.

3 Why AI/ML is not ideal in Kernel Space

- The kernel operates with strict resource constraints to maintain system stability and performance:
 - Memory Management: The kernel does not have access to virtual memory like user-space programs. Memory allocations in the kernel use physical memory (e.g., `kmalloc`), which is much more constrained. Large ML models (e.g., neural network weights) may exhaust this limited memory. Dynamically allocating large buffers (like those required for storing weights, activations, or intermediate outputs during inference) can cause kernel instability.
 - No Floating-Point Operations in Kernel: Floating-point operations (critical for most ML computations) are disabled in the kernel to avoid contextswitching overhead and interference with hardware's FPU (floatingpoint unit). Implementing custom floating-point logic in the kernel would be both slow and error-prone.
- The kernel's primary job is to manage system-level processes (e.g., scheduling, I/O, memory) efficiently. Performing computationally intensive tasks like ML inference could:
 - Block Critical Processes: Kernel functions run in privileged mode and preempt other processes. If inference takes significant time, it could block essential system processes (e.g., interrupt handlers), causing system instability or performance degradation.
 - Violate Real Time Constraints: Real-time systems rely on predictable kernel behavior. Long-running ML inference could lead to unpredictable latencies.
- Performing inference in the kernel introduces complexity in debugging and error handling:
 - Crashes Are Critical: Errors in user-space programs (e.g., segmentation faults) are isolated to that process, while kernel crashes (e.g., NULL pointer dereferences or memory leaks) can freeze or crash the entire system.
 - Lack of Debugging Tools: Debugging kernel modules is more challenging than debugging user-space code. Developers must rely on limited tools (e.g., `dmesg` or `printk`) to diagnose issues, making development slower and more error-prone.
- Modern ML inference frameworks (e.g., PyTorch, TensorFlow, or ONNX Runtime) leverage highly optimized libraries like cuDNN, Intel MKL, or OpenBLAS, which are unavailable in the kernel. Implementing equivalent optimizations in the kernel is impractical and inefficient. SIMD (Single Instruction, Multiple Data) instructions like AVX or GPUs cannot be efficiently utilized in the kernel due to hardware constraints and architectural limitations.
- Running inference in the kernel could introduce security vulnerabilities:
 - Kernel Privileges: Kernel modules run with full system privileges. Any bug in the inference logic could become a potential attack vector for privilege escalation or system compromise.
 - Unvalidated Inputs: User inputs (e.g., model input data) passed to the kernel must be sanitized to prevent attacks like buffer overflows or injecting malicious code.
- Most modern ML frameworks rely on parallelism (via multi-threading or GPUs) to accelerate inference. The Linux kernel lacks the mechanisms to

manage such parallel execution efficiently. For example: GPU drivers are user-space libraries. Threads and process-level parallelism are strictly managed by the kernel but cannot themselves run efficiently inside the kernel.

4 Methodology

The architectural foundations begin with a simple computation kernel module that will be implemented as a Loadable Kernel Module (LKM). This LKM [Appendix A] registers a custom syscall to perform basic arithmetic operations (e.g., addition, subtraction, multiplication, and division). This basic LKM is then extended to handle more involved computations including tensor operations leading to deep learning inference. The tensorial LKM [Appendix B] handle tensors (multidimensional arrays) and support basic model-like computations, such as matrix multiplication or summation across tensors. This lays the foundations for integrating more complex deep learning operations later.

The implementation strategy involves developing the computational LKM using the Linux kernel module programming interface, specifically leveraging the `module_init()` and `module_exit()` macros for lifecycle management. The custom syscall is registered using the `sys_call_table` modification technique, with careful consideration of write-protection mechanisms through CR0 register manipulation. For tensor operations, we implement a custom memory management scheme using `kmalloc()` for dynamic allocation of tensor structures, with careful attention to alignment requirements for optimal SIMD instruction utilization. The tensor operations are optimized using architecture-specific intrinsics (e.g., AVX-512 for x86_64 platforms) to achieve maximum computational efficiency. Memory mapping between user and kernel space is facilitated through `get_user_pages()` and `kmap()` functions, enabling zero-copy data transfer where possible. The tensor operations are implemented using a blocked algorithm approach to optimize cache utilization, with block sizes determined empirically based on the target architecture’s cache hierarchy. Error handling is implemented comprehensively using return code propagation and appropriate cleanup mechanisms to maintain system stability, particularly in cases of memory allocation failures or invalid tensor dimensions.

5 Why LKMs are useful?

Loadable Kernel Modules (LKMs) can handle real-time data acquisition and preprocessing for modalities like: image preprocessing for vision models, audio signal filtering for speech models, sensor fusion for robotics, etc.

Example: Using an LKM to handle raw camera data and output tensors for a vision model. Certain AGI applications (e.g., robotics or autonomous systems) require real-time responses. LKMs can provide deterministic execution for critical decision-making. For example using an LKM for collision detection in robotics. LKMs can act as an interface to hardware accelerators like GPUs or TPUs for specific tasks in deep learning. Example: Use LKMs to offload vision inference tasks to a GPU in a low-latency pipeline. Different LKMs can serve as specialized nodes for processing, like vision LKM handles image recognition models, language LKM processes NLP models, and audio LKM manages speech recognition. These nodes can then communicate through high-speed channels (e.g., kernel networking or shared memory). Loadable Kernel Modules are specialized, but their outputs must be unified in higher-level abstractions. There is a need of a user-space orchestrator to fuse these modalities into a unified representation. AGI requires systems capable of adapting to new data and continually improving. Kernel modules are static at runtime (except for reloading), making them ill-suited for dynamic updates. However, the operating system can use an LKM to handle model inference. and offload dynamic learning tasks to user-space components. So, using LKMs for deterministic operations (e.g., running trained models), but use distributed systems or multi-agent architectures to model emergent behavior would lay a strong foundation for future systems design. User-space orchestrator manages the interaction between LKMs, data routing between modalities and participates in high-level decision-making. While LKMs can contribute to AGI development by providing real-time, deterministic modules for specialized tasks, AGI requires a higher-level, integrated, and adaptable framework that goes beyond kernel-level design. LKMs are best suited for preprocessing data, low-latency, modular computations while supporting hardware acceleration.

A sample AGI Decision Engine:

- **Cognitive Reasoning:** Executes symbolic reasoning and decision-making. For example, A reinforcement learning policy to decide robotic actions.
- **Planning:** Long-term strategies (e.g., achieving goals across multiple modalities). For example combining a text command (“pick up the red ball”) with visual recognition to control a robot.
- **Emergent Behavior:** Built on self-supervised learning and interaction between subsystems. Example: Language and vision subsystems collaborating to interpret ambiguous scenarios.

A Data-Flow Example:

- *Input*: A camera provides an image, a microphone provides speech, and a LIDAR sensor provides spatial data.
- *Kernel Space*: Vision LKM identifies objects and locations in the image. Audio LKM transcribes speech into text. Sensor LKM computes distances and generates a spatial map.
- *User-space Orchestrator*: Collects and synchronizes outputs from kernel space. It converts outputs into a unified multi-modal representation.
- *Multi-modal Fusion Layer*: Integrates the visual, auditory, and spatial data into a unified tensor. Context: "A person is standing 3 meters away, asking for help."
- *AGI Decision Engine*: Decides an action: "Approach the person and respond verbally." and outputs a robotic control command or text response.

6 Performance Optimization

Before extending the kernel module for handling real model weights like those from ONNX or TensorFlow Lite (TFLite), we explore the performance optimization [Appendix C] for the current implementation provided in Appendix B which suffers due to single-threaded execution, excessive memory usage and input/output overhead. Matrix operations, especially multiplication, scale poorly with larger tensors when running in a single thread and copying tensors between user and kernel space for large operations incurs significant performance overhead. Kernel memory is crucial and limited. Excessive use can lead to system instability.

Linux Kernel Modules (LKMs) offer significant potential for advancing Artificial General Intelligence (AGI) systems by providing low-level, high-performance capabilities tailored to specific computational tasks. For modality-specific preprocessing, LKMs can directly handle real-time data acquisition and transformation, making them ideal for use cases such as image preprocessing for vision models, audio signal filtering for speech recognition, or sensor fusion in robotics. By processing raw inputs at the kernel level and outputting formatted tensors or optimized data structures, LKMs minimize overhead and ensure seamless integration with downstream AI pipelines. In the context of real-time decision-making, they enable deterministic execution of time-critical operations, such as collision detection in robotics or navigation in autonomous systems, where rapid response times are paramount. Furthermore, hardware acceleration is another domain where LKMs

excel, acting as interfaces to specialized accelerators like GPUs or TPUs. By offloading computation-heavy tasks, such as vision inference, to these devices through low-latency pipelines, LKMs optimize performance and reduce bottlenecks. Additionally, they can function as distributed AGI components, each tailored to a specific modality: a vision LKM could handle image recognition, a language LKM could process NLP tasks, and an audio LKM could manage speech-to-text conversion. These modular components can communicate via high-speed kernel networking or shared memory, forming a cohesive, distributed system that integrates diverse capabilities. This modular and efficient use of LKMs positions them as vital building blocks for the scalable and responsive infrastructure needed in AGI systems.

Leveraging Linux Kernel Modules (LKMs) for Artificial General Intelligence (AGI) involves creating specialized modules tailored to handle modality-specific tasks, forming the core of an efficient and modular architecture. Modality-specific LKMs can be developed for distinct areas such as vision, language, and audio. For vision tasks, LKMs can handle operations like image classification and object detection, while language modules can support sentence parsing and essential Natural Language Processing (NLP) tasks. Audio-related LKMs may focus on speech-to-text conversion or noise filtering. These specialized LKMs are coordinated by an orchestrator layer built in userspace, which manages interactions between different LKMs, facilitates data routing between modalities, and handles high-level decision-making to ensure smooth system integration. To optimize resource utilization, lightweight preprocessing tasks and low-latency inference are handled by LKMs, while computationally heavy operations, including large-scale model training or inference, are offloaded to user-space applications or distributed systems. A distributed LKM framework can be implemented by leveraging kernel-level networking technologies, such as Netlink or RDMA, to distribute the processing load across multiple nodes, each dedicated to a specific modality. This framework enables scalable, efficient handling of complex AGI tasks, ensuring that the system is both flexible and powerful enough to accommodate diverse types of data and operations.

The kernel threads are used to parallelize computations like matrix multiplication. The kernel memory allocation can be minimized by using DMA buffers and reducing tensor copies. Large buffers can be allocated in the DMA memory area (`dma_alloc_coherent`). We should operate directly on user-space memory when feasible using pinned pages (`get_user_pages`). For large tensors, the optimization uses shared memory (e.g.,

through mmap) to allow both user and kernel space to access same buffer. In the syscall, it maps tensor data from user space into kernel space. This reduces overhead by avoiding **copy_from_user/copy_to_user**. One example for this is to use **remap_pfn_range** for shared memory in the LKM.

7 System Architecture

The UML diagram below describes the interaction between various kernel and user-space components in a distributed processing framework designed for an AGI system, with a focus on Linux Kernel Modules (LKMs) and their orchestration.

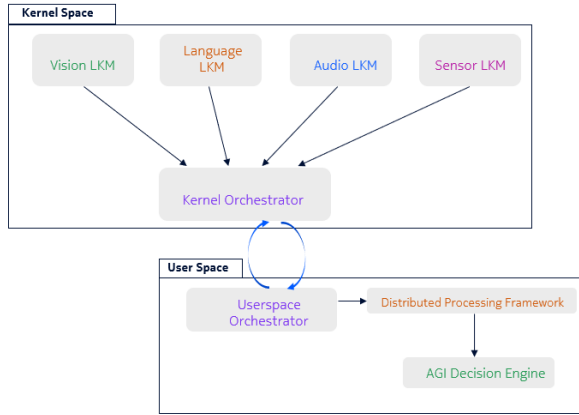


fig1. System Architecture 1

In the Kernel Space, there are several modality-specific LKMs:

- **Vision LKM:** Handles tasks related to image processing and vision, such as object detection or image classification.
- **Language LKM:** Focuses on language-related tasks, such as sentence parsing, part-of-speech tagging, and other NLP tasks.
- **Audio LKM:** Processes audio input, possibly converting speech to text or performing noise filtering.
- **Sensor LKM:** Deals with data from various physical sensors, potentially involving sensor data preprocessing or sensor fusion.

All these LKMs connect to the Kernel Orchestrator. The Kernel Orchestrator is responsible for managing the interactions between the various LKMs, routing data between them, and ensuring that the kernel space handles each modality's specific tasks efficiently. In the User Space, the User Orchestrator connects to the

Kernel Orchestrator. This orchestrator acts as an intermediary between the kernel-space LKMs and the higher-level user-space components, allowing for coordination of activities, routing of data, and high-level decision-making. The Distributed Processing Framework in the user space is a key element in enabling scalable processing. It interacts with the AGI Decision Engine, which is responsible for high-level decisions, learning, and reasoning. The distributed framework connects to the Kernel Orchestrator to manage the distribution of tasks across multiple nodes. These nodes handle specific modalities and interact with the kernel to process data locally, offloading computations from the main system. The Distributed Processing Framework also allows the system to distribute processing workloads efficiently across different computational nodes. This could involve distributing tasks like image classification, language processing, or sensor data analysis across multiple machines or specialized hardware. Each node can handle a specific modality (e.g., vision, language, audio) and communicate with others through the orchestrator layer.

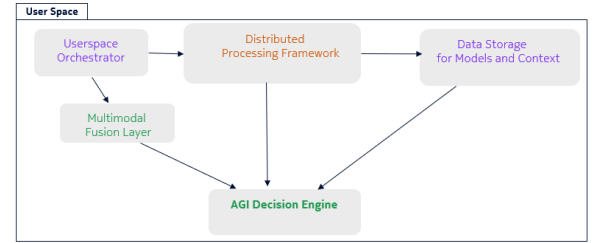


fig2. System Architecture 2

7.1 Workflow and Connections:

- **Kernel Orchestrator to User Orchestrator:** The Kernel Orchestrator provides the required low-level processing and data flow management, while the User Orchestrator manages the system's high-level coordination and decision-making. The two communicate to ensure smooth data flow and orchestration between kernel and user spaces.
- **Distributed Processing Framework to Kernel Orchestrator:** Framework communicates with the Kernel Orchestrator to manage the distribution of tasks, ensuring that processing is efficiently spread across multiple nodes. This enables parallel processing and scalability.
- **Distributed Processing Framework to AGI Decision Engine:** The Distributed Processing Framework interacts with the AGI Decision Engine to send processed data or results for higher-level decision-making. The AGI Decision Engine uses these re-

sults for tasks like reasoning, learning, and planning, driving the AGI system’s behavior.

In summary, the diagram illustrates a hybrid system where the kernel-space LKMs handle modality-specific tasks, and the user-space orchestrator and distributed framework manage high-level decision-making and distribution of tasks across computational nodes. This structure enables efficient processing of different data types and facilitates scaling across multiple systems, ultimately powering an AGI system.

7.2 Kernel-to-User Communication

Efficient kernel-to-user communication is critical for transferring data outputs from LKMs (kernel space) to user-space orchestrators. We use shared memory as our primary approach. Shared memory reduces the overhead of copying data between kernel and user space. We’ll use `remap_pfn_range` to share memory.

8 System Architecture Enhancements

The system architecture incorporates comprehensive enhancements across multiple domains, integrating robust fault tolerance mechanisms, dynamic load balancing, and standardized communication protocols. A fault tolerance mechanism could be implemented for continuous state replication across distributed nodes, periodic checkpoint management for rapid recovery, and real-time failure detection through heartbeat mechanisms, complemented by automated failover procedures and distributed operational logging. The dynamic load balancing component can optimize resource utilization through real-time monitoring of CPU, memory, and I/O metrics, employing predictive analytics with machine learning models for proactive resource allocation, and implementing adaptive scheduling algorithms tailored to different modality-specific workloads (vision, language, audio, sensor). A standardized inter-node communication protocol could establish a unified framework featuring consistent message formatting, quality of service controls, and robust compression and encryption mechanisms, while maintaining version control for backward compatibility. The enhanced architecture integrates comprehensive security measures including authentication mechanisms, access control systems, and audit trails, along with performance optimizations through strategic caching, pipeline optimization, and resource reservation systems. A sophisticated monitoring and management layer should provide comprehensive logging, performance metrics collection, and administrative controls, all working in concert to enhance the system’s reliability, efficiency, and

manageability while preserving its core AGI processing capabilities.

9 Conclusion

The evolution of Loadable Kernel Modules from basic system extensions to sophisticated computational frameworks represents a significant advancement in high-performance computing architecture. By combining robust fault tolerance mechanisms, dynamic load balancing, and standardized communication protocols with kernel-level computational capabilities, this approach offers a powerful paradigm for next-generation distributed systems. The prototype implementation demonstrates that careful consideration of security boundaries, memory management, and hardware optimization can enable efficient kernel-space computations while maintaining system stability and reliability. This architecture’s ability to handle diverse computational modalities while ensuring fault tolerance and optimal resource utilization positions it as a promising foundation for advanced computing applications, particularly in domains requiring real-time processing and high reliability. The successful integration of these components, from basic arithmetic operations to complex tensor computations, validates the feasibility of using LKMs as building blocks for sophisticated computational frameworks, opening new possibilities for future research and development in distributed computing systems.

References

- [1] Giorgio Buttazzo. “Can We Trust AI-Powered Real-Time Embedded Systems?” In: *Third Workshop on Next Generation Real-Time Embedded Systems (NG-RES 2022)*. Ed. by Marko Bertogna, Federico Terraneo, and Federico Reghenzani. Vol. 98. Open Access Series in Informatics (OASIs). Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, 1:1–1:14. ISBN: 978-3-95977-221-1. DOI: 10 . 4230 / OASIs . NG - RES . 2022 . 1. URL: <https://drops.dagstuhl.de/entities/document/10.4230/OASIs.NG-RES.2022.1>.
- [2] Edoardo Cittadini et al. “Supporting AI-powered real-time cyber-physical systems on heterogeneous platforms via hypervisor technology”. In: *Real-Time Syst.* 59.4 (July 2023), pp. 609–635. ISSN: 0922-6443. DOI: 10.1007/s11241-023-09402-4. URL: <https://doi.org/10.1007/s11241-023-09402-4>.
- [3] Bryan Henderson. *Loadable Kernel Modules*. Accessed: 2025-01-13. 2025.

- [4] Mike Krinkin. *Kernel modules, device drivers and Device Tree*. Accessed: 2025-01-13. 2025.
- [5] Barun Kumar Parichha. “Performance Analysis of Process Using Loadable Kernel Module (LKM)”. In: *2015 Fifth International Conference on Communication Systems and Network Technologies*. 2015, pp. 1338–1343. DOI: 10.1109/CSNT.2015.184.
- [6] Yuhei Suzuki et al. “Real-Time GPU Resource Management with Loadable Kernel Modules”. In: *IEEE Transactions on Parallel and Distributed Systems* 28.6 (2017), pp. 1715–1727. DOI: 10.1109/TPDS.2016.2630697.
- [7] CS UCLA. *Dynamic Modules*. Accessed: 2025-01-13. 2025.
- [8] Yifan Zhang et al. *Integrating Artificial Intelligence into Operating Systems: A Comprehensive Survey on Techniques, Applications, and Future Directions*. 2024. arXiv: 2407.14567 [cs.OS]. URL: <https://arxiv.org/abs/2407.14567>.

Appendix

A Simple Loadable Kernel Module Implementation

Here is a quick tutorial demonstrating how to write an LKM to handle custom computation and to integrate a custom syscall into the kernel. A simple user-space program is also given to test the syscall.

Input: Two integers (a and b) and an operation code (e.g., 0=add, 1=subtract).

Output: The result of the operation.

Error Handling: Includes division-by-zero checks and proper error propagation to the user space.

Step1: Kernel Module Implementation

Listing 1: simple_compute.c

```

1 #include <linux/kernel.h>
2 #include <linux/module.h>
3 #include <linux/init.h>
4 #include <linux/syscalls.h>
5 #include <linux/uaccess.h>
6 #include <linux/slab.h>
7
8 // Module metadata
9 MODULE_LICENSE("GPL");
10 MODULE_AUTHOR("Software Architect GPT");
11 MODULE_DESCRIPTION("LKM for Simple Computation");
12 MODULE_VERSION("1.0");
13
14 // Arithmetic operations
15 #define OP_ADD 0
16 #define OP_SUBTRACT 1
17 #define OP_MULTIPLY 2
18 #define OP_DIVIDE 3
19
20 // Custom computation function
21 static int compute(int a, int b, int op, int *result) {
22     switch (op) {
23         case OP_ADD:
24             *result = a + b;
25             break;
26         case OP_SUBTRACT:
27             *result = a - b;
28             break;
29         case OP_MULTIPLY:
30             *result = a * b;
31             break;

```

```

32     case OP_DIVIDE:
33         if (b == 0)
34             return -EINVAL; // Division by zero error
35         *result = a / b;
36         break;
37     default:
38         return -EINVAL; // Invalid operation
39 }
40 return 0; // Success
41 }
42
43 // Custom syscall implementation
44 asmlinkage long sys_simple_compute(int a, int b, int op, int __user *result) {
45     int kernel_result;
46     int ret;
47
48     // Perform computation
49     ret = compute(a, b, op, &kernel_result);
50     if (ret < 0)
51         return ret; // Return error to user space
52
53     // Copy result to user space
54     if (copy_to_user(result, &kernel_result, sizeof(kernel_result)))
55         return -EFAULT;
56
57     return 0; // Success
58 }
59
60 // Module initialization
61 static int __init simple_compute_init(void) {
62     printk(KERN_INFO "Simple Compute LKM: Module loaded.\n");
63     return 0;
64 }
65
66 // Module cleanup
67 static void __exit simple_compute_exit(void) {
68     printk(KERN_INFO "Simple Compute LKM: Module unloaded.\n");
69 }
70
71 module_init(simple_compute_init);
72 module_exit(simple_compute_exit);

```

Step2: Register the Custom Syscall To make this function available as a syscall, kernel's syscall table must be modified by declaring the syscall in *include/linux/syscalls.h*. After declaring, it should be added to *arch/x86/entry/syscalls/syscall_64.tbl*. The following entry should be added as shown below:

Listing 2: Modifying the Syscall Table

```

1 548      common      sys_simple_compute      sys_simple_compute

```

And then, declare the syscall in *include/linux/syscalls.h*

Listing 3: Declaring the Syscall

```

1 asmlinkage long sys_simple_compute(int a, int b, int op, int __user *result);

```

Step3: Build and Load the Kernel Module

Listing 4: Makefile

```

1 obj-m += simple_compute.o

```



```

2
3 all:
4     make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
5
6 clean:
7     make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean

```

Compile the kernel module using the *make* command which will produce *simple_compute.ko* file and then use the **insmod** command to load the module.

Listing 5: Modifying the Syscall Table

```

1 sudo insmod simple_compute.ko

```

Step4: Test the Syscall

Listing 6: Test Syscall

```

1 #define _GNU_SOURCE
2 #include <unistd.h>
3 #include <sys/syscall.h>
4 #include <stdio.h>
5 #include <errno.h>
6
7 #define SYS_SIMPLE_COMPUTE 548
8
9 int main() {
10     int a = 10, b = 5, op = 0; // Example: Addition
11     int result;
12
13     long ret = syscall(SYS_SIMPLE_COMPUTE, a, b, op, &result);
14     if (ret == 0) {
15         printf("Result: %d\n", result);
16     } else {
17         perror("Syscall failed");
18     }
19
20     return 0;
21 }

```

Compile and test the program given above to verify the functionality of custom syscall. If the syscall fails, you will see an error message.

B Loadable Kernel Module (LKM) for Tensor Operations

Listing 7: tensor_compute.c

```

1 #include <linux/kernel.h>
2 #include <linux/module.h>
3 #include <linux/init.h>
4 #include <linux/syscalls.h>
5 #include <linux/uaccess.h>
6 #include <linux/slab.h>
7
8 // Module metadata
9 MODULE_LICENSE("GPL");
10 MODULE_AUTHOR("Software Architect GPT");
11 MODULE_DESCRIPTION("LKM for Tensor Operations");
12 MODULE_VERSION("1.1");

```

```

13
14 // Operation codes
15 #define OP_SUM 0
16 #define OP_MATRIX_MULT 1
17
18 // Custom tensor operation function
19 static int tensor_compute(const int *tensor1, size_t rows1, size_t cols1,
20                          const int *tensor2, size_t rows2, size_t cols2,
21                          int *result, size_t *result_rows, size_t *result_cols, int op)
22 {
23     size_t i, j, k;
24
25     if (op == OP_SUM) {
26         // Element-wise sum: Ensure same dimensions
27         if (rows1 != rows2 || cols1 != cols2)
28             return -EINVAL;
29
30         *result_rows = rows1;
31         *result_cols = cols1;
32
33         for (i = 0; i < rows1; i++) {
34             for (j = 0; j < cols1; j++) {
35                 result[i * cols1 + j] = tensor1[i * cols1 + j] + tensor2[i * cols1 + j];
36             }
37         }
38     } else if (op == OP_MATRIX_MULT) {
39         // Matrix multiplication: cols1 must match rows2
40         if (cols1 != rows2)
41             return -EINVAL;
42
43         *result_rows = rows1;
44         *result_cols = cols2;
45
46         for (i = 0; i < rows1; i++) {
47             for (j = 0; j < cols2; j++) {
48                 int sum = 0;
49                 for (k = 0; k < cols1; k++) {
50                     sum += tensor1[i * cols1 + k] * tensor2[k * cols2 + j];
51                 }
52                 result[i * cols2 + j] = sum;
53             }
54         }
55     } else {
56         return -EINVAL; // Unsupported operation
57     }
58
59     return 0; // Success
60 }
61
62 // Custom syscall implementation
63 asmlinkage long sys_tensor_compute(const int __user *tensor1, size_t rows1, size_t cols1,
64                                   const int __user *tensor2, size_t rows2, size_t cols2,
65                                   int __user *result, size_t __user *result_rows, size_t
66                                   __user *result_cols, int op) {
67     int *kernel_tensor1 = NULL, *kernel_tensor2 = NULL, *kernel_result = NULL;
68     size_t kernel_result_rows, kernel_result_cols;
69     int ret;
70
71     // Allocate memory for tensors in kernel space

```

```

70     kernel_tensor1 = kmalloc(rows1 * cols1 * sizeof(int), GFP_KERNEL);
71     kernel_tensor2 = kmalloc(rows2 * cols2 * sizeof(int), GFP_KERNEL);
72     kernel_result = kmalloc(rows1 * cols2 * sizeof(int), GFP_KERNEL); // Max possible
        size
73
74     if (!kernel_tensor1 || !kernel_tensor2 || !kernel_result) {
75         ret = -ENOMEM;
76         goto cleanup;
77     }
78
79     // Copy tensors from user space to kernel space
80     if (copy_from_user(kernel_tensor1, tensor1, rows1 * cols1 * sizeof(int)) ||
81         copy_from_user(kernel_tensor2, tensor2, rows2 * cols2 * sizeof(int))) {
82         ret = -EFAULT;
83         goto cleanup;
84     }
85
86     // Perform computation
87     ret = tensor_compute(kernel_tensor1, rows1, cols1,
88                         kernel_tensor2, rows2, cols2,
89                         kernel_result, &kernel_result_rows, &kernel_result_cols, op);
90
91     if (ret < 0)
92         goto cleanup;
93
94     // Copy result back to user space
95     if (copy_to_user(result, kernel_result, kernel_result_rows * kernel_result_cols *
96         sizeof(int)) ||
97         copy_to_user(result_rows, &kernel_result_rows, sizeof(size_t)) ||
98         copy_to_user(result_cols, &kernel_result_cols, sizeof(size_t))) {
99         ret = -EFAULT;
100     }
101
102 cleanup:
103     kfree(kernel_tensor1);
104     kfree(kernel_tensor2);
105     kfree(kernel_result);
106     return ret;
107 }
108
109 // Module initialization
110 static int __init tensor_compute_init(void) {
111     printk(KERN_INFO "Tensor Compute LKM: Module loaded.\n");
112     return 0;
113 }
114
115 // Module cleanup
116 static void __exit tensor_compute_exit(void) {
117     printk(KERN_INFO "Tensor Compute LKM: Module unloaded.\n");
118 }
119
120 module_init(tensor_compute_init);
121 module_exit(tensor_compute_exit);

```

Listing 8: User-Space Program for SysCalling Tensor Operations

```

1 #define _GNU_SOURCE
2 #include <unistd.h>
3 #include <sys/syscall.h>
4 #include <stdio.h>

```

```

5 #include <errno.h>
6 #include <string.h>
7 #include <stdlib.h>
8
9 #define SYS_TENSOR_COMPUTE 548
10
11 #define OP_SUM 0
12 #define OP_MATRIX_MULT 1
13
14 void print_matrix(const int *matrix, size_t rows, size_t cols) {
15     for (size_t i = 0; i < rows; i++) {
16         for (size_t j = 0; j < cols; j++) {
17             printf("%d ", matrix[i * cols + j]);
18         }
19         printf("\n");
20     }
21 }
22
23 int main() {
24     int tensor1[] = {1, 2, 3, 4, 5, 6}; // 2x3 matrix
25     int tensor2[] = {7, 8, 9, 10, 11, 12}; // 2x3 matrix for sum or 3x2 matrix for
        multiplication
26     int result[100]; // Placeholder for the result matrix
27     size_t result_rows, result_cols;
28
29     // Operation: Matrix Multiplication
30     long ret = syscall(SYS_TENSOR_COMPUTE, tensor1, 2, 3, tensor2, 3, 2, result, &
        result_rows, &result_cols, OP_MATRIX_MULT);
31     if (ret == 0) {
32         printf("Result of Matrix Multiplication (%lux%lu):\n", result_rows, result_cols);
33         print_matrix(result, result_rows, result_cols);
34     } else {
35         perror("Syscall failed");
36     }
37
38     return 0;
39 }

```

Listing 9: Compile and Test

```

1 make
2 sudo insmod tensor_compute.ko
3 gcc -o test_tensor test_tensor.c
4 ./test_tensor

```

Listing 10: Multi-Threading Matrix Multiplication

```

1 #include <linux/kthread.h>
2 #include <linux/completion.h>
3
4 #define MAX_THREADS 8
5
6 struct thread_data {
7     const int *tensor1;
8     const int *tensor2;
9     int *result;
10    size_t start_row;
11    size_t end_row;
12    size_t cols1;

```

```

13     size_t cols2;
14     size_t cols_result;
15     struct completion *completion;
16 };
17
18 static int thread_matrix_mult(void *data) {
19     struct thread_data *tdata = (struct thread_data *)data;
20     size_t i, j, k;
21
22     for (i = tdata->start_row; i < tdata->end_row; i++) {
23         for (j = 0; j < tdata->cols2; j++) {
24             int sum = 0;
25             for (k = 0; k < tdata->cols1; k++) {
26                 sum += tdata->tensor1[i * tdata->cols1 + k] *
27                     tdata->tensor2[k * tdata->cols2 + j];
28             }
29             tdata->result[i * tdata->cols_result + j] = sum;
30         }
31     }
32
33     complete(tdata->completion);
34     return 0;
35 }
36
37 // Multi-threaded matrix multiplication
38 static int parallel_matrix_mult(const int *tensor1, size_t rows1, size_t cols1,
39                                const int *tensor2, size_t rows2, size_t cols2,
40                                int *result, size_t *result_rows, size_t *result_cols) {
41     struct task_struct *threads[MAX_THREADS];
42     struct thread_data tdata[MAX_THREADS];
43     struct completion completions[MAX_THREADS];
44     size_t rows_per_thread;
45     size_t i;
46
47     if (cols1 != rows2)
48         return -EINVAL;
49
50     *result_rows = rows1;
51     *result_cols = cols2;
52     rows_per_thread = rows1 / MAX_THREADS + (rows1 % MAX_THREADS != 0);
53
54     for (i = 0; i < MAX_THREADS; i++) {
55         size_t start_row = i * rows_per_thread;
56         size_t end_row = min(start_row + rows_per_thread, rows1);
57
58         if (start_row >= rows1)
59             break;
60
61         init_completion(&completions[i]);
62         tdata[i] = (struct thread_data){
63             .tensor1 = tensor1,
64             .tensor2 = tensor2,
65             .result = result,
66             .start_row = start_row,
67             .end_row = end_row,
68             .cols1 = cols1,
69             .cols2 = cols2,
70             .cols_result = cols2,
71             .completion = &completions[i],

```

```

72     };
73
74     threads[i] = kthread_run(thread_matrix_mult, &tdata[i], "matrix_mult_thread");
75     if (IS_ERR(threads[i]))
76         return -EFAULT;
77 }
78
79 for (i = 0; i < MAX_THREADS && tdata[i].start_row < rows1; i++) {
80     wait_for_completion(&completions[i]);
81 }
82
83 return 0;
84 }

```

Listing 11: Handling Weights Dynamically

```

1 static int load_weights(const int __user *user_weights, size_t weight_size) {
2     int *kernel_weights = kmalloc(weight_size * sizeof(int), GFP_KERNEL);
3     if (!kernel_weights)
4         return -ENOMEM;
5
6     if (copy_from_user(kernel_weights, user_weights, weight_size * sizeof(int))) {
7         kfree(kernel_weights);
8         return -EFAULT;
9     }
10
11     // Use kernel_weights for inference...
12     kfree(kernel_weights);
13     return 0;
14 }

```

Listing 12: Shared Memory for Kernel-to-User Communication

```

1 #include <linux/module.h>
2 #include <linux/mm.h>
3 #include <linux/uaccess.h>
4 #include <linux/slab.h>
5 #include <linux/fs.h>
6
7 #define SHARED_MEM_SIZE (PAGE_SIZE)
8
9 static char *shared_mem;
10 static int device_open(struct inode *inode, struct file *file) {
11     return 0;
12 }
13
14 static int device_mmap(struct file *file, struct vm_area_struct *vma) {
15     if (remap_pfn_range(vma, vma->vm_start,
16         virt_to_phys(shared_mem) >> PAGE_SHIFT,
17         vma->vm_end - vma->vm_start,
18         vma->vm_page_prot)) {
19         return -EAGAIN;
20     }
21     return 0;
22 }
23
24 static const struct file_operations fops = {
25     .open = device_open,
26     .mmap = device_mmap,

```

```

27 };
28
29 static int __init shared_mem_init(void) {
30     shared_mem = kmalloc(SHARED_MEM_SIZE, GFP_KERNEL);
31     if (!shared_mem)
32         return -ENOMEM;
33
34     strcpy(shared_mem, "Kernel Shared Memory Example");
35
36     if (register_chrdev(240, "shared_mem", &fops) < 0) {
37         kfree(shared_mem);
38         return -EBUSY;
39     }
40
41     printk(KERN_INFO "Shared memory module loaded\n");
42     return 0;
43 }
44
45 static void __exit shared_mem_exit(void) {
46     unregister_chrdev(240, "shared_mem");
47     kfree(shared_mem);
48     printk(KERN_INFO "Shared memory module unloaded\n");
49 }
50
51 module_init(shared_mem_init);
52 module_exit(shared_mem_exit);
53
54 MODULE_LICENSE("GPL");
55 MODULE_AUTHOR("Software Architect GPT");
56 MODULE_DESCRIPTION("Shared Memory Kernel-to-User Communication");

```

Listing 13: User-space Program for Shared Memort Communication

```

1  #include <fcntl.h>
2  #include <stdio.h>
3  #include <sys/mman.h>
4  #include <string.h>
5
6  #define SHARED_MEM_SIZE (4096)
7
8  int main() {
9      int fd = open("/dev/shared_mem", O_RDWR);
10     if (fd < 0) {
11         perror("Open failed");
12         return -1;
13     }
14
15     char *shared_mem = mmap(NULL, SHARED_MEM_SIZE, PROT_READ | PROT_WRITE, MAP_SHARED, fd
16     , 0);
17     if (shared_mem == MAP_FAILED) {
18         perror("mmap failed");
19         close(fd);
20         return -1;
21     }
22
23     printf("Read from kernel: %s\n", shared_mem);
24     munmap(shared_mem, SHARED_MEM_SIZE);
25     close(fd);

```

```
26     return 0;  
27 }
```